# Laporan Tugas Kecil 2

# IF2211 Strategi Algoritma

## Pemanfaatan Algoritma Divide and Conquer dalam Voxelization Objek 3D menggunakan Octree


Disusun oleh:

13524073 Keisha Rizka Syofyani

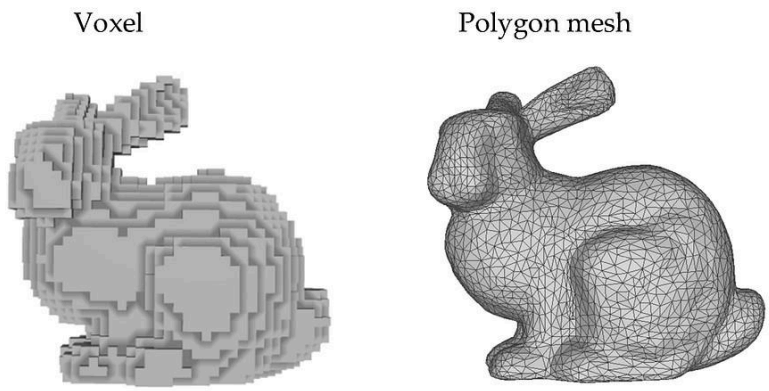
13524091 Vara Azzara Ramli Pulukadang

**Sekolah Teknik Elektro dan Informatika**

**Institut Teknologi Bandung**

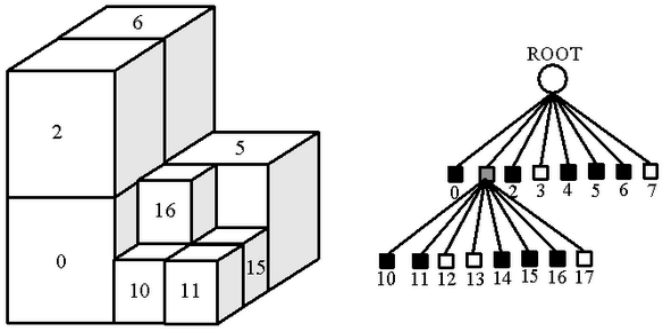

**2026**

# Deskripsi Tugas




Gambar 1 Visualisasi *voxelization*
Sumber: https://www.mdpi.com/1424-8220/22/18/7040

*Voxelization* merupakan proses konversi objek tiga dimensi yang umum menjadi sebuah model yang tersusun dari kubus-kubus kecil seperti di Minecraft. Proses konversi ini dilakukan dengan memanfaatkan struktur data **Octree**, yaitu sebuah *tree* yang setiap percabangan terdiri dari 8 anak. Melalui *Octree*, sebuah ruang kubus 3D dapat dipotong menjadi 8 kubus berbeda, yang masing-masing dapat dipotong lagi menjadi 8 kubus yang lebih kecil.

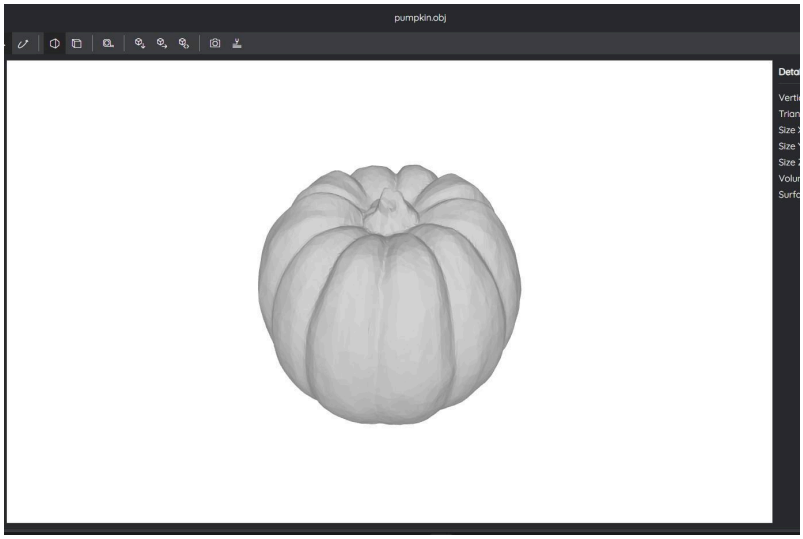



Gambar 2 Visualisasi *Octree* terhadap 3D Model
Sumber:
https://www.researchgate.net/figure/a-Octants-b-Octree-Structure-c-Linear-Octree-Each-of-the-cubes-has-a-locational-code_fig1_221115507

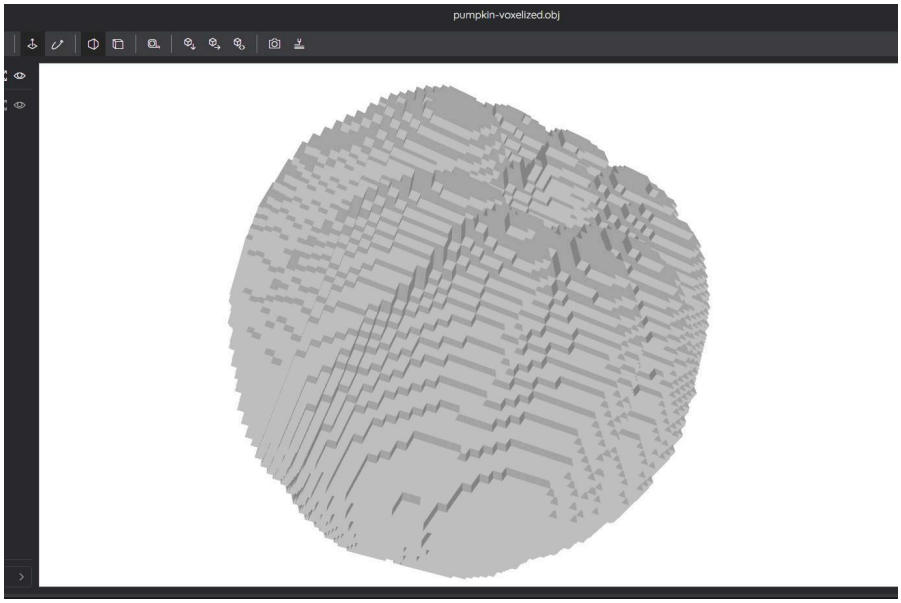
Pada tugas kecil ini kami diminta untuk dapat mengimplementasikan sebuah program yang dapat mengkonversi model .obj yang umum menjadi model .obj yang terdiri dari *voxel* dengan memanfaatkan properti ***Octree***. Program melakukan konversi model dengan menggunakan algoritma ***divide and conquer.***

## Ilustrasi Kasus

Diberikan sebuah model 3D sebagai berikut.




Di bawah adalah hasil perubahan.



# Implementasi Algoritma Divide and Conquer

## 1. Algoritma Divide and Conquer

Algoritma divide and conquer merupakan algoritma yang digunakan di dalam penugasan kali ini. Adapun konsep utama algoritma ini adalah membagi persoalan menjadi beberapa subpersoalan yang lebih kecil lalu menyelesaikan masing-masing subpersoalan secara rekursif dan memperoleh solusi akhir berdasarkan hasil dari subpermasalahan tersebut.

Pada penugasan ini, kami diminta untuk membagi ruang 3D secara efisien dan hanya fokus ke bagian yang mengandung objek dengan mengimplementasikan struktur data octree.

Pada konsep octree, kubus dapat dibagi menjadi delapan bagian lebih kecil. Setiap pembagian kubus akan dilakukan pengecekan apakah kubus tersebut berpotongan dengan permukaan objek yang diberikan dalam bentuk triangle mesh. Pengecekan ini dilakukan menggunakan pendekatan *Axis-Aligned Bounding Box (AABB)*. Jika kubus tidak berpotongan dengan objek, maka kubus tersebut tidak perlu ditelusuri lebih lanjut (*pruning*). Sebaliknya, jika terdapat potensi perpotongan, maka kubus akan dipecah kembali menjadi delapan kubus yang lebih kecil.

Proses ini dilakukan secara rekursif hingga mencapai kedalaman maksimum (maximum depth) yang telah ditentukan. Semakin besar nilai depth dari octree maka ukuran voxel yang dihasilkan akan semakin kecil sehingga menghasilkan representasi objek yang lebih detail. Sebaliknya, depth yang lebih kecil akan menghasilkan voxel yang lebih besar dan representasi yang lebih kasar. Voxel yang dihasilkan dari leaf node inilah yang digunakan sebagai representasi diskrit dari *polygon mesh* yang diberikan sebagai input.

## 2. Cara Kerja Program

Program ini diimplementasikan menggunakan bahasa pemrograman C++. Pemilihan bahasa ini didasarkan pada beberapa pertimbangan. Selain telah dipelajari sebelumnya, C++ juga memiliki performa yang tinggi serta kontrol yang baik terhadap penggunaan memori, sehingga cocok untuk menangani komputasi geometri tiga dimensi dan pembangunan struktur data octree yang kompleks. Selain itu, C++ menyediakan Standard Template Library (STL) yang memudahkan dalam pengelolaan struktur data seperti penyimpanan vertex dan face. C++ juga mendukung multithreading, sehingga dapat dimanfaatkan untuk meningkatkan performa program, khususnya pada proses rekursif dalam pembangunan octree melalui pendekatan concurrency.

Struktur data utama yang digunakan adalah OctreeNode, yang menyimpan informasi berupa bounding box (AABB), pointer ke delapan child node, status leaf (isLeaf), serta kedalaman node dalam tree (depth). Secara umum, alur program dimulai dengan menerima input dari user yang terdiri atas file objek (.obj) yang berisi data vertice (v) dan faces (f) dari *polygon mesh* objek serta nilai *maximum depth* dari octree yang akan dibangun. Program kemudian akan membaca file objek dan menyimpan data triangle dan vertices objek. Program menghitung *bounding box*

(AABB) dari seluruh scene objek dan membuat kubus awal sebagai root *octree node* menyesuaikan ukuran objek dan mencakup seluruh objek.

Tahap berikutnya adalah algoritma utama divide and conquer melalui struktur *octree*. Program akan melakukan pengecekkan *overlap* antara *node octree* dengan AABB dari *triangle mesh* objek. pengecekan overlap antara bounding box node octree dengan triangle menggunakan metode triangle-AABB intersection (*Separating Axis Theorem* / SAT). Metode ini digunakan untuk menentukan apakah suatu segitiga berpotongan dengan *bounding box node octree*. Jika tidak terjadi *overlap*, maka node tersebut tidak perlu ditelusuri lebih lanjut (*pruning*) dan mencatat jumlah pruneCount. Sebaliknya, jika terdapat potensi *overlap*, maka kubus akan dibagi menjadi delapan bagian yang lebih kecil (*subdivide*) dan proses dilakukan secara rekursif hingga mencapai *maximum depth* yang sudah didefinisikan sebelumnya. Pada setiap tahap subdividingnya, dilakukan filtering terhadap triangle yang relevan terhadap masing-masing child node, yakni triangle yang berpotongan dengan box child yang akan lanjut ke proses rekursi, sedangkan yang tidak berpotongan akan dilakukan pruning

Setelah proses pembangunan octree selesai, program melakukan traversal untuk mengumpulkan seluruh node leaf (isLeaf == true). Setiap leaf node kemudian dikonversi menjadi sebuah voxel berbentuk kubus berdasarkan nilai bounding box dari node tersebut. Setiap voxel direpresentasikan oleh 8 vertex dan 12 segitiga (faces), sesuai dengan representasi mesh pada format objek. Dari rangkaian proses ini, dihasilkan:

- Jumlah *voxel* yang terbentuk : jumlah leaf node dari maximum depth objek yang berpotongan dengan objek → count isLeaf == true
- Jumlah *vertex* yang terbentuk : titik-titik kubus → 8 * jumlah voxel
- Jumlah *faces* : jumlah sisi segitiga dari kubus → 12 * jumlah voxel
- Statistik jumlah node *octree* di setiap *depth*
- Statistik jumlah node yang tidak perlu ditelusuri (*pruning*) pada setiap *depth*

Program ini juga menghitung lama waktu program berjalan untuk menghasilkan voxel dalam satuan detik dengan menggunakan pustaka chrono. Kemudian program akan menyimpan hasil objek yang sudah diubah menjadi voxel ke sebuah file objek (.obj) baru. Program juga dapat menampilkan informasi-informasi melalui CLI berdasarkan nilai-nilai yang sudah didapatkan selama proses sebelumnya. Adapun informasi yang akan ditampilkan diantaranya sebagai berikut.

- Banyaknya *voxel* yang terbentuk
- Banyaknya *ver*tex yang terbentuk
- Banyaknya *faces* yang terbentuk
- Statistik *node octree* yang terbentuk
- Statistik *node* yang **tidak perlu** ditelusuri
- Kedalaman *octree*
- Lama waktu program berjalan
- Path dimana file .obj disimpan

Secara umum, kompleksitas waktu bergantung pada jumlah triangle dan kedalaman octree. Dengan adanya pruning dan filtering, kompleksitas dapat ditekan dibandingkan brute force traversal seluruh ruang.

Pada penugasan ini, kami mengerjakan salah satu bonus, yakni penggunaan concurrency guna meningkatkan performa program. Konsep concurrency digunakan untuk memberikan optimasi pada pembangunan *octree* dengan memanfaatkan kemampuan *multithreading* bahasa C++ untuk menjalankan beberapa proses secara paralel. Concurrency dilakukan pada tahap rekursif proses *subdivide node octree*. Proses ini dilakukan dengan membuat agar *octree node* melakukan *subdividing* pada masing-masing *child*-nya secara paralel yang ditetapkan pada depth awal (depth < 2) untuk menghindari overhead penggunaan thread yang berlebihan. Hal ini karena setiap *child node* bersifat independen sehingga proses bersamaan tidak mempengaruhi hasil akhir. Dengan memanfaatkan *concurrency* ini, waktu eksekusi program dapat diefisienkan, terutama pada *maximum depth* yang sangat besar. Untuk menghindari race condition pada proses concurrency, variabel penghitung seperti nodeCount dan pruneCount diimplementasikan menggunakan tipe data atomic, sehingga operasi increment dapat dilakukan secara thread-safe. Pada implementasinya, concurrency yang kami buat ini menggunakan fasilitas multithreading dari bahasa C++, yaitu std::async, yang memungkinkan eksekusi fungsi secara asynchronous.

## 3. Struktur Data

| Nama Struct | Atribut | Tipe Data | Keterangan |
|---|---|---|---|
| ParseOBJ | triangles | Array of Triangle | Menyimpan daftar seluruh wajah (*faces*) objek 3D dalam bentuk segitiga |
| | vertices | Array of vec3 | Menyimpan koordinat titik-titik pembentuk objek. |
| vec3 | x, y, z | real | Representasi titik atau vektor dalam ruang 3D (Cartesian). |
| AABB | min, max | vec3 | *Axis-Aligned Bounding Box*: Kotak pembatas terkecil yang mengurung seluruh objek (digunakan sebagai batas luar Octree). |

4. Definisi Kelas
   Nama kelas : OctreeNode

Atribut :

- bounds : AABB
- Array of children : OctreeNode
- isLeaf : boolean
- depth : integer

Method :

- OctreeNode(bounds : AABB, depth : integer) : Konstruktor
- ~OctreeNode() : Destruktor

## 5. Pseudocode Program

1) Pre-Processing / Parser Objek

**File Parser.cpp:**

```
function parseOBJ(filePath : string, result : ParsedOBJ,
errorMessage : string) → boolean
{Fungsi membaca file objek dan mengekstrak data koordinat
titik dan face segitiga sesuai struktur data program}
      if (file.isOpen() = false) then
            errorMessage ← ""Tidak bisa membuka file " +
filepath”
            → false

lineNumber ← 0

while(getLine(file, line) = true) do
      lineNumber ← lineNumber + 1

ss >> token

depend on token
      token = “v” :
            if(ss >> v.x >> v.y >> v.z = false) then
                  errorMessage ← (“format vertex tidak vaid
      di baris ”, lineNumber)
                  → false
            vertices.push_back(v)
      token = “f” :
            if(ss >> i >> j >> k = false) then
                  errorMessage ← (“format faces tidak vaid
      di baris ”, lineNumber)
                  → false
      n ← vertices.size()
```

```
        if (i < 1 or j < 1 or k < 1 or i > n or j > n or k >
        n) then
             erorrMessage ← ("Indeks face tidak valid di
        baris ", lineNumber);
             → false;

        tri.v0 ← vertices[i - 1]
        tri.v1 ← vertices[j - 1]
        tri.v2 ← vertices[k - 1]
        result.triangles.push_back(tri)

if(vertices.empty() = true) then
     errorMessage ← ("File tidak mengandung vertex")
     → false
if(result.triangles.empty() = true) then
     errorMessage ← ("File tidak mengandung face")
     → false
result.vertices ← vertices
→ true
```

**File main.cpp:**

```
{input file objek}
input(fileName)
filePath ← "../test/Objek/" + fileName
if(fileCheck.isOpen() = false) then
     cer ← ("Error: Tidak ditemukan file", fileName)
open(fileName)
if(open(fileName) = false) then
     cerr ← ("error : tidak ditemukan file", fileName)

{input maxDepth}
input(maxDepth)
if(parseOBJ(filePath, parsed, errorMsg) = false) then
     err ← errMsg

{Parse dan tampilkan data file objek}
output(parsed.vertices.size())
output(parsed.triangle.size())
bounds ← computeBounds(parsed.vertices)
output(bounds.min.x, bounds.min.y, bounds.min.z)
output(bounds.max.x, bounds.max.y, bounds.max.z)

sideX ← bounds.max.x - bound.min.x
sideY ← bounds.max.y - bound.min.y
sideZ ← bounds.max.z - bound.min.z
```

```
output(sideX, sideY, sideZ)
```

2) Algoritma Divide and Conquer

**File octree.cpp:**

```
function OctreeNode(bounds : AABB, depth : integer) →
OctreeNode
{Konstruktor OctreeNode}
     i traversal [0..7]:
          children[i] ← nullptr

function ~OctreeNode(bounds : AABB, depth : integer) →
OctreeNode
{Destruktor OctreeNode}
     i traversal [0..7]:
               Delete children[i]

function subdivide (box : AABB) → array of AABB
{Pembagi ruang AABB menjadi 8 sub yang lebih kecil}
     center ← aabb_center(box)
     min ← box.min
     max ← box.max
     → {{min, center}, {center.x, min.y, min.z}, {min.x,
     center.y, min.z}, {center.x, max.y, center.z}},
     {{center.x, center.y, min.z}, {max.x, max.y,
     center.z}}, {{min.x, min.y, center.z}, {center.x,
     center.y, max.z}}, {{center.x, min.y, center.z},
     {max.x, center.y, max.z}}, {{min.x, center.y,
     center.z}, {center.x, max.y, max.z}}, {center, max}}

buildOctree(triagle : array of Triangle, bounds : AABB,
depth : integer, maxDepth : integer, nodeCount : integer, pruneCount
: integer) → OctreeNode
{Pembangun Octree secara rekursif : Algoritma Divide and
Conquer}
     nodeCount[depth] ← nodeCount[depth] + 1
     hasInteraction ← false
     i traversal [tri : triangles]:
          if(triangleIntersectsAABB(tri, bounds) then
               hasIntersection ← true
     if(hasIntersection = false) then
          pruneCount[depth] ← pruneCount[depth] + 1
          → nullptr

     node ← OctreeNode(bounds, depth)
     if(depth = maxDepth) then
```

```
                    node.isLeaf ← true
                    → node
               childBounds ← subdivide(bounds)

               i traversal [0..7]
                    if(depth < 2 and depth < maxDepth - 1) then
                         futures[i] ← async(launch.async, [=,
triangles, nodeCount, pruneCount]:
                              i traversal [tri : triangles]:
                                   if(triangleIntersectsAABB(tri,
childBounds[i]) then
                                   childTriangles.push_back(tri)

                              → buildOctree(childTriangles,
childBounds[i], depth + 1, maxDepth, nodeCount, pruneCount)
                    else
                         i traversal [tri : triangles]:
                              if(triangleIntersectsAABB(tri,
childBounds[i]) then
                                   childTriangles.push_back(tri)
                         node.children[i] ←
buildOctree(childTriangles, childBounds[i], depth + 1, maxDepth,
nodeCount, pruneCount)

          i traversal [0..7]:
               if(depth < 2 and depth < maxDepth - 1) then
                    Node.children[i] ← futures[i].get()

          → node
```

**File main.cpp:**

```
{Start Timer}
start ← high_resolutions_clock.now()

{Buat root}
root ← buildOctree(parsed.triangles, bounds, 0, maxDepth,
nodeCount.data(), pruneCount.data())

{Proses utama}
collectLeafNodes(root, leaves)

{End Timer}
end ← high_resolutions_clock.now()
```

3) Write Objek dan Output Generation

**File writer.cpp**

```
procedure collectLeafNodes(node : OctreeNode, leaves :
array of OctreeNode)
{Melakukan traversal pada octree untuk mengumpulkan leaf
node pada voxel}
     if(node = false) then return
     if(node.isLeaf) then
          leaves.push(node)
          return
     i traversal [0..7]:
          collectLeafNodes(node->children[i], leaves)

function generateCubeVertices(box : AABB) → array of vec3
{menghasilkan 8 vertex yang membentuk kubus AABB pada leaf
node}
     min = box.min
     max = box.max
     → {{min.x, min.y, min.z},
        {max.x, min.y, min.z},
        {min.x, max.y, min.z},
        {max.x, max.y, min.z},
        {min.x, min.y, max.z},
        {max.x, min.y, max.z},
        {min.x, max.y, max.z},
        {max.x, max.y, max.z}}

function generateCubeFaces(offset : integer) → array of
(array of 3 integer)
{menghasilkan 12 face segitiga pada kubus yang sudah
terbentuk}
     → {offset+1, offset+2, offset+3}, {offset+2,
offset+4, offset+3},
        {offset+5, offset+6, offset+7}, {offset+6,
offset+8, offset+7},
        {offset+1, offset+2, offset+5}, {offset+2,
offset+6, offset+5},
        {offset+2, offset+4, offset+6}, {offset+4,
offset+8, offset+6},
        {offset+4, offset+3, offset+8}, {offset+3,
offset+7, offset+8},
        {offset+3, offset+1, offset+7}, {offset+1,
offset+5, offset+7}
    }
```

```
procedure writeOBJ(fileName : string, leaves : array of
OctreeNode)
{menyusun seluruh vertex dan faces pada sebuah file dengan
format objek sebagai output}
     open(fileName)
     vertexOffset ← 0
     i traversal [node : leaves]:
          vertices ← generateCubeVertices(node.bounds)
          i traversal [v : vertices]:
               write(v.x, v.y, v.z)
          i traversal [f : faces]:
               write(f[0], f[1], f[2])

     vertexOffset ← vertexOffset + 8

close(fileName)
```

**File main.cpp**

```
{Simpan seluruh nilai yang ingin ditampilkan}
duration ← end - start
voxelCount ← leaves.size()
vertexCount ← voxelCount * 8
faceCount ← voxelCount * 12
outputPath ← filePath + "_output.obj"

writeOBJ(outputPath, leaves)

{Output di CLI}
output(voxelCount)
output(vertexCount)
output(faceCount)
i traversal [0..maxDepth]:
     output(i, nodeCount[i])

i traversal [0..maxDepth]:
     output(i, pruneCount[i])
output(maxDepth)
output(duration)
output(outputPath)
```

# Pengujian Program

| No | Input | Output |
|---|---|---|
| 1 | File teapot.obj<br><br>maxDepth = 2 | Masukkan nama file objek (.obj) pada path ../test/Objek/ : tea pot.obj<br>Masukkan maximum depth octree : 5<br>Vertices  : 530<br>Triangles : 1024<br>AABB min  : (-84.4453, -86.4664, -92.4691)<br>AABB max  : (98.2769, 96.2558, 90.2531)<br>Cube side : 182.722 (should equal 182.722 and 182.722)<br><br>========== HASIL VOXELIZATION OBJEK ==========<br>Jumlah voxel yang terbentuk : 1748 voxel<br>Jumlah vertex yang terbentuk : 13984 vertex<br>Jumlah faces yang terbentuk : 20976 faces<br><br>Statistik Node Octree :<br>Depth-0 : 1 node<br>Depth-1 : 8 node<br>Depth-2 : 64 node<br>Depth-3 : 192 node<br>Depth-4 : 864 node<br>Depth-5 : 3344 node<br><br>Statistik Node Octree Pruned :<br>Depth-0 : 0 node pruned<br>Depth-1 : 0 node pruned<br>Depth-2 : 40 node pruned<br>Depth-3 : 84 node pruned<br>Depth-4 : 446 node pruned<br>Depth-5 : 1596 node pruned<br><br>Kedalaman octree : 5<br>Processing time : 0.0177959 seconds<br>Output objek berhasil disimpan pada file :<br>../test/Objek/teapot_output.obj |
| 2 | File pumpkin.obj<br><br>maxDepth = 2 | Masukkan nama file objek (.obj) pada path ../test/Objek/ : pum pkin.obj<br>Masukkan maximum depth octree : 7<br>Vertices  : 5002<br>Triangles : 10000<br>AABB min  : (-42.1959, -38.702, -149.588)<br>AABB max  : (36.9486, 40.4425, -70.4435)<br>Cube side : 79.1445 (should equal 79.1445 and 79.1445) |

| | | |
|---|---|---|
| | | ========== HASIL VOXELIZATION OBJEK ==========<br>Jumlah voxel yang terbentuk : 74750 voxel<br>Jumlah vertex yang terbentuk : 598000 vertex<br>Jumlah faces yang terbentuk : 897000 faces<br><br>Statistik Node Octree :<br>Depth-0 : 1 node<br>Depth-1 : 8 node<br>Depth-2 : 64 node<br>Depth-3 : 448 node<br>Depth-4 : 2216 node<br>Depth-5 : 8984 node<br>Depth-6 : 37008 node<br>Depth-7 : 149096 node<br><br>Statistik Node Octree Pruned :<br>Depth-0 : 0 node pruned<br>Depth-1 : 0 node pruned<br>Depth-2 : 8 node pruned<br>Depth-3 : 171 node pruned<br>Depth-4 : 1093 node pruned<br>Depth-5 : 4358 node pruned<br>Depth-6 : 18371 node pruned<br>Depth-7 : 74346 node pruned<br><br>Kedalaman octree : 7<br>Processing time : 0.0888645 seconds<br>Output objek berhasil disimpan pada file :<br>../test/Objek/pumpkin_output.obj |
| 3 | File line.obj<br><br>maxDepth = 2 | Masukkan nama file objek (.obj) pada path ../test/Objek/ : lin e.obj<br>Masukkan maximum depth octree : 10<br>Vertices  : 3<br>Triangles : 1<br>AABB min  : (2.21249, -0.992723, -0.969115)<br>AABB max  : (2.42722, -0.777999, -0.75439)<br>Cube side : 0.214724 (should equal 0.214724 and 0.214724)<br><br>========== HASIL VOXELIZATION OBJEK ==========<br>Jumlah voxel yang terbentuk : 230327 voxel<br>Jumlah vertex yang terbentuk : 1842616 vertex<br>Jumlah faces yang terbentuk : 2763924 faces<br><br>Statistik Node Octree :<br>Depth-0 : 1 node<br>Depth-1 : 8 node |

| | | |
|---|---|---|
| | | Depth-2 : 56 node<br>Depth-3 : 112 node<br>Depth-4 : 256 node<br>Depth-5 : 736 node<br>Depth-6 : 2296 node<br>Depth-7 : 8144 node<br>Depth-8 : 30600 node<br>Depth-9 : 118280 node<br>Depth-10 : 464920 node<br><br>Statistik Node Octree Pruned :<br>Depth-0 : 0 node pruned<br>Depth-1 : 1 node pruned<br>Depth-2 : 42 node pruned<br>Depth-3 : 80 node pruned<br>Depth-4 : 164 node pruned<br>Depth-5 : 449 node pruned<br>Depth-6 : 1278 node pruned<br>Depth-7 : 4319 node pruned<br>Depth-8 : 15815 node pruned<br>Depth-9 : 60165 node pruned<br>Depth-10 : 234593 node pruned<br><br>Kedalaman octree : 10<br>Processing time : 0.125226 seconds<br>Output objek berhasil disimpan pada file :<br>../test/Objek/line_output.obj |
| 4 | File cow.obj<br><br>maxDepth = 6 | Masukkan nama file objek (.obj) pada path ../test/Objek/ : cow .obj<br>Masukkan maximum depth octree : 6<br>Vertices  : 4583<br>Triangles : 5804<br>AABB min  : (-4.44584, -5.66062, -5.22196)<br>AABB max  : (5.99809, 4.7833, 5.22196)<br>Cube side : 10.4439 (should equal 10.4439 and 10.4439)<br><br>========== HASIL VOXELIZATION OBJEK ==========<br>Jumlah voxel yang terbentuk : 5627 voxel<br>Jumlah vertex yang terbentuk : 45016 vertex<br>Jumlah faces yang terbentuk : 67524 faces<br><br>Statistik Node Octree :<br>Depth-0 : 1 node<br>Depth-1 : 8 node<br>Depth-2 : 64 node<br>Depth-3 : 160 node<br>Depth-4 : 704 node |

| | | |
|---|---|---|
| | | Depth-5 : 2656 node<br>Depth-6 : 10896 node<br><br>Statistik Node Octree Pruned :<br>Depth-0 : 0 node pruned<br>Depth-1 : 0 node pruned<br>Depth-2 : 44 node pruned<br>Depth-3 : 72 node pruned<br>Depth-4 : 372 node pruned<br>Depth-5 : 1294 node pruned<br>Depth-6 : 5269 node pruned<br><br>Kedalaman octree : 6<br>Processing time : 0.0208644 seconds<br>Output objek berhasil disimpan pada file :<br>../test/Objek/cow_output.obj |
| 5 | File kosong (tidak valid) | Error: Tidak ditemukan file kosong.obj |
| 6 | Input depth tidak valid | Masukkan nama file objek (.obj) pada path ../test/Objek/ : cow<br>.obj<br>Masukkan maximum depth octree : a<br>Error: Input tidak valid! Masukkan angka bulat nonnegatif.<br><br>Masukkan maximum depth octree : 5 |
| 7 | Format isi file tidak valid : tidak terdeteksi adanya vertex | Masukkan nama file objek (.obj) pada path ../test/Objek/ : val<br>idation_test.obj<br>Masukkan maximum depth octree : 3<br>Error: File tidak mengandung vertex |
| 8 | Format isi file tidak valid : tidak terdeteksi adanya faces | Masukkan nama file objek (.obj) pada path ../test/Objek/ :<br>validation_test.obj<br>Masukkan maximum depth octree : 3<br>Error: File tidak mengandung face |
| 9 | Format isi file tidak valid : format penulisan vertex tidak valid | Masukkan nama file objek (.obj) pada path ../test/Objek/ :<br>validation_test.obj<br>Masukkan maximum depth octree : 3<br>Error: Format vertex tidak valid di baris 2 |
| 10 | Format isi file tidak valid : format penulisan faces tidak valid | Masukkan nama file objek (.obj) pada path ../test/Objek/ :<br>validation_test.obj<br>Masukkan maximum depth octree : 3<br>Error: Format face tidak valid di baris 4 |

# Lampiran

1. **Link Repository Github**
   https://github.com/keirzka/Tucil2_13524073_13524091.git

2. **Checklist Tugas**

| No | Poin | Ya | Tidak |
|---|---|---|---|
| 1 | Program berhasil di kompilasi tanpa kesalahan | v | |
| 2 | Program berhasil dijalankan | v | |
| 3 | Hasil konversi .obj terdiri dari kubus-kubus kecil | v | |
| 4 | Program informasi-informasi atau report | v | |
| 5 | Program dapat menerima masukan .obj, dan mengembalikan output | v | |
| 6 | [Bonus] Program menggunakan *concurrency* | v | |
| 7 | [Bonus] Program dapat menampilkan visualisasi .obj | | |

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

Keisha Rizka Syofyani

13524073

Vara Azzara Ramli Pulukadang

13524091